



Python in Research, Part 3

Modelling Epidemics

Most of us have been affected by the fear of H1N1 (swine flu). As with any epidemic, public health organisations need to model this pandemic too, in order to be prepared with preventive or precautionary actions. In this article, you will look at how Scipy can be used to model an epidemic using the simple SIR model.

The basic idea is that the population consists of three groups—the susceptible, the infected and those who have recovered. It is reasonable to assume that the rate at which susceptible people are infected will be proportional to the possible pairs of susceptible and infected populations, i.e., their product. It is also reasonable to assume that the rate at which people recover will be proportional to the infected population. Finally, the rate of change in the infected group will be the difference between the infection rate and the recovery rate. The SIR model can, thus, be represented as a set of simple ordinary differential equations (assuming that there are no births or deaths):

- $ds/dt = -bs(t)i(t)$ # rate of change amongst the susceptible
- $di/dt = bs(t)i(t) - ki(t)$ # rate of change amongst the infected
- $dr/dt = ki(t)$ # rate of change of those who've recovered

If the factors b and k are known, and the initial values of the susceptible, infected and recovered populations are also known, the above equations can be integrated and a solution found.

An elementary example

Suppose 10 people in a city of a million are infected. The infectious period of a flu typically lasts for five days; so, an estimated 20 per cent of the infected cases recover each day.

Usually, the values of s , i and r are

normalised so that the sum of the three is 1. In this case, the ratio of b to k is indicative of the number of people infected by an infected person. Assuming that each infected person infects another two, what will happen in the next 100 days?

Here's how you can find out by substituting the values in the following equation:

$$b = 2 * k = 0.4$$

$$k = 0.2$$

The Scipy package includes an *integrate* module, which can be used to compute the susceptible, infected and recovered populations for each succeeding day, for as many days as needed.

```
import scipy as np
from scipy import integrate
def dif_eq(V,t,b,k):
    ...
    Compute the derivatives for the differential
    equations
    V = current values of [Susceptible, Infected,
    Recovered]
    ...
    dVdt = np.zeros(3)
    dVdt[0] = -b*V[0]*V[1]
    dVdt[1] = b*V[0]*V[1] - k*V[1]
    dVdt[2] = k*V[1]
    return dVdt
P = 1e6 # a million
IO = 10/P # 10 people are infected
SO = 1 - IO # Susceptible population
RO = 0 # Initial Recovered
k = 0.2 # Infection lasts 5 days
```